

The lux `apply_patch` Atomicity Boundary: a Z Specification

Formal model of the all-or-nothing patch discipline

July 2026

1 Introduction

An agent patches a single Element-ABC element by sending a mapping of field names to new values. `Element.apply_patch` applies the entries in order, each through a `_set_<key>` setter, and promises *all-or-nothing*: if any setter rejects its value, every field the patch already touched is restored and the element is left exactly as it was before the patch began. A composed table also has an *observer* — the filtered table’s Hub-side model — that folds each selection write into a full selection spanning filter-hidden rows. The property that must hold is that *a rejected patch produces no observable change at all*: neither the element’s fields nor any observer’s state may differ from their pre-patch values, and a patch that succeeds must leave every observer reflecting the patch’s final values.

The same class of defect — a rejected patch leaving an observable trace — recurred across two fix/review rounds:

1. the table’s selection lived in a *mutable* composed model, so the shallow `dict(vars(self))` snapshot captured the model *reference*; a setter mutated that same object in place, and the rollback restored a reference to an already-mutated model — the field was not restored;
2. the selection setter notified observers *inside* the patch, so a composed model folded the intermediate selection into its full selection before a later key was rejected; the rollback restored the element’s fields but could not un-fire the notification — the observer’s state was not restored.

Round 1 was closed by making the composed selection model immutable, so each setter reassigns a new model and the shallow snapshot captures the old value, not a shared reference. Round 2 is closed by *deferring* observer notification: a setter queues its notifications, `apply_patch` fires them only after the whole patch commits, and a rejected patch fires nothing. Both fixes survive here as fidelity variants (Section 10) that reintroduce each defect to confirm the model still detects it. Empirical round-by-round fixing kept missing an interleaving, so this document models the boundary as a finite state machine over a bounded patch, letting ProB enumerate every patch shape exhaustively.

2 Basic Types

$[KEY, OBS]$

`KEY` is the set of field names a patch may set. `OBS` is the set of registered observers; the filtered table’s model is one such observer. Two of each suffice to exhibit a patch that sets one watched field and then fails on a second field, and an observer that watches the mutated field; the carriers are bounded by ProB’s `DEFAULT_SETSIZE`.

3 Free Types

$GOODVAL ::= a \mid b$

The values a field may validly hold. Two distinct values are enough to show a field *changing* and then being restored: the pre-patch value and a new one a patch installs.

$PHASE ::= idle \mid applying$

The patch lifecycle. **idle**: no patch in progress; the observable invariant must hold. **applying**: a patch is mid-flight, so deferred notifications may legitimately not yet be reflected.

A patch step either sets a known field to a valid value or names a value the setter rejects:

$$STEP ::= setK \langle\langle KEY \times GOODVAL \rangle\rangle \mid badK \langle\langle KEY \rangle\rangle$$

setK: set field **k** to valid value **g** (the setter succeeds). **badK**: set field **k** to a rejected value (the setter raises, triggering rollback). Each constructor is an injection, and their ranges partition **STEP**, so a step is a set or a reject, never both, and the test $head\ queue \in \text{ran } setK$ distinguishes them. Two projections recover a step's target and (for a set) its value:

$$\begin{array}{|l} \hline stepKey : STEP \rightarrow KEY \\ stepVal : STEP \rightarrow GOODVAL \\ \hline \forall k : KEY; g : GOODVAL \bullet \\ \quad stepKey(setK(k, g)) = k \wedge stepVal(setK(k, g)) = g \\ \forall k : KEY \bullet stepKey(badK\ k) = k \\ \text{dom } stepVal = \text{ran } setK \end{array}$$

stepKey is total — every step names a target. **stepVal** is defined exactly on the **setK** steps, because only a successful set carries an installed value.

Each observer watches exactly one field; a selection observer watches the selection field. The assignment is a fixed parameter of the system, left unconstrained so ProB explores every mapping — including the ones where an observer watches the field a patch mutates, which is what makes the invariant detect a field left unrestored:

$$\mid \quad watches : OBS \rightarrow KEY$$

The patch is a bounded sequence of steps; the bound keeps the carrier finite so ProB can enumerate every ordering:

$$\begin{array}{|l} \hline maxlen : \mathbb{N} \\ \hline maxlen = 3 \end{array}$$

4 State

The state is flat: one schema, six components. Its predicate carries only the length bound on the queue — a structural constraint true in every design. The atomicity property (Section 8) is deliberately *absent* here, so a violating state stays reachable and ProB can find it.

$$\begin{array}{|l} \hline Patch \\ \hline phase : PHASE \\ fields : KEY \rightarrow GOODVAL \\ snap : KEY \rightarrow GOODVAL \\ observed : OBS \rightarrow GOODVAL \\ pending : \mathbb{P} OBS \\ queue : \text{seq } STEP \\ \hline \#queue \leq maxlen \end{array}$$

fields is the element's current field values — one value per field. **snap** is the rollback snapshot taken when a patch begins: the values **apply_patch** restores on failure. **observed** records, for each observer, the field value it has folded in — the abstraction of the filtered model's full selection, which is a function of the selection value at the moment it was notified. **pending** is the set of observers with a *deferred* notification queued by the in-flight patch, fired only at commit. **queue** is the unprocessed tail of the current patch.

The observer's real fold is a set expression — $(\text{full} \setminus \text{visible}) \cup (\text{write} \cap \text{visible})$ — that preserves hidden selections. That detail is orthogonal to atomicity: what a rejected patch must not do is leave the observer having captured *any* value it should not have. The model therefore abstracts the fold to “the field value the observer was last notified about”, because the defect is precisely that the observer captured an intermediate value a rollback could not withdraw.

5 Initialization

<i>Init</i>
<i>Patch'</i>
$phase' = idle$ $fields' = (\lambda k : KEY \bullet a)$ $snap' = (\lambda k : KEY \bullet a)$ $observed' = (\lambda o : OBS \bullet a)$ $pending' = \emptyset$ $queue' = \langle \rangle$

A single initial state: no patch in progress, every field holds **a**, every observer has folded **a** (so each observer already reflects its watched field), nothing is pending, and the queue is empty.

6 Building and Beginning a Patch

An agent builds a patch one step at a time while the element is idle, then begins applying it. Building step-by-step — rather than supplying a whole sequence at once — keeps every input a single **STEP** over a finite carrier, so ProB enumerates the patch shapes without trying to enumerate arbitrary sequences. **Enqueue** appends one step, up to the length bound; any step is admissible, including a **badK** that will later cause a rollback:

<i>Enqueue</i>
$\Delta Patch$
$p? : STEP$
$phase = idle$ $\#queue < maxlen$ $queue' = queue \frown \langle p? \rangle$ $phase' = phase$ $fields' = fields$ $snap' = snap$ $observed' = observed$ $pending' = pending$

BeginApply starts the built patch: it takes the rollback snapshot — a *value* copy of the current fields, unmodified during the idle build — and flips to **applying**. The empty patch is admissible; it simply commits with nothing to do:

<i>BeginApply</i>
$\Delta Patch$
$phase = idle$ $phase' = applying$ $snap' = fields$ $pending' = \emptyset$ $queue' = queue$ $fields' = fields$ $observed' = observed$

7 Applying a Patch

Each step processes the head of the queue. A successful set installs the value and *queues* the notification for every observer of that field, but does not fire it — notification is deferred to commit:

ApplyKey

$\Delta Patch$

$phase = applying$
 $queue \neq \langle \rangle$
 $head\ queue \in \text{ran } setK$
 $fields' = fields \oplus \{stepKey(head\ queue) \mapsto stepVal(head\ queue)\}$
 $pending' = pending \cup$
 $\quad \{o : OBS \mid watches\ o = stepKey(head\ queue)\}$
 $queue' = tail\ queue$
 $phase' = phase$
 $snap' = snap$
 $observed' = observed$

When the queue is empty, the patch commits: every deferred notification fires, folding each pending observer's watched field — now at its final committed value — into the observer's state:

CommitPatch

$\Delta Patch$

$phase = applying$
 $queue = \langle \rangle$
 $observed' = observed \oplus$
 $\quad \{o : pending \bullet o \mapsto fields(watches\ o)\}$
 $phase' = idle$
 $fields' = fields$
 $snap' = snap$
 $pending' = \emptyset$
 $queue' = \langle \rangle$

A rejected step aborts the whole patch: the fields are restored from the snapshot, the deferred notifications are discarded unfired, and the rest of the batch is dropped. Because nothing was ever fired, the observers are untouched — the rejected patch produces no observable change:

AbortPatch

$\Delta Patch$

$phase = applying$
 $queue \neq \langle \rangle$
 $head\ queue \in \text{ran } badK$
 $fields' = snap$
 $observed' = observed$
 $pending' = \emptyset$
 $queue' = \langle \rangle$
 $phase' = idle$
 $snap' = snap$

The three apply schemas partition every reachable **applying** state: whenever the queue is non-empty the head is a **setK** (**ApplyKey**) or a **badK** (**AbortPatch**), and whenever it is empty **CommitPatch** is enabled. With **BeginApply** enabled in every **idle** state, the machine never gets stuck (Section 9, deadlock-freedom).

8 The Invariant

One safety property must hold across all reachable states. It is stated here as a predicate over **Patch**; it is *not* placed in the **Patch** schema, because a predicate placed there would be enforced as a guard on every successor, silently disabling any operation that would break it and thereby masking the very defect this model exists to catch.

Invariant AT — observable atomicity. Whenever no patch is in progress, every observer reflects exactly its watched field’s current value:

$$phase = idle \Rightarrow (\forall o : OBS \bullet observed\ o = fields\ (watches\ o)).$$

This one predicate captures both halves of the all-or-nothing contract, from opposite sides:

- *After a successful patch*, `CommitPatch` sets each pending observer to its watched field’s final value, and an observer whose field the patch never touched is unchanged alongside an unchanged field — so `observed o = fields (watches o)` for every observer. Observers reflect the patch’s final values.
- *After a rejected patch*, `AbortPatch` restores every field from the value snapshot and fires no notification, so both sides return to their pre-patch values and remain equal. The rejected patch produced no observable change.

A design that breaks either fix breaks this equality at an `idle` state: round 2 leaves `observed o` ahead of a restored `fields`, and round 1 leaves `fields` ahead of an untouched `observed`. The invariant catches the divergence whichever side moved.

9 Verification

Type-checking is a `fuzz` pass:

```
fuzz -t docs/patch_atomicity.tex
```

Invariant AT is checked by asking ProB whether its negation is *reachable*. A goal that is not found means the property holds on every reachable state:

```
# Invariant AT (must report: goal NOT found)
probccli docs/patch_atomicity.tex -model_check -nodead \
  -goal "phase=idle & {o|o:OBS & observed(o) /= fields(watches(o))} /= {}" \
  -p DEFAULT_SETSIZE 2
```

Deadlock-freedom is the state-space search with no goal; the machine is kept live by `BeginApply` in every idle state and by an apply schema while a patch is in progress, so a correct design never deadlocks:

```
# Deadlock-freedom (must report: no deadlock, no invariant violation)
probccli docs/patch_atomicity.tex -model_check -p DEFAULT_SETSIZE 2
```

At `DEFAULT_SETSIZE 2` both searches are exhaustive at the full patch length (`maxlen = 3`): every reachable state is visited and every operation covered. Both are re-run at `DEFAULT_SETSIZE 3` with the patch length capped at 2 — enough to exhibit the two length-two defect traces of Section 10 — to confirm the result is not an artefact of the two-element carrier; that search too visits every state with no counter-example. Raising both bounds at once (three-element carriers with length-three patches) is unnecessary: each defect trace is a length-two patch, and the two bounds are independent.

10 Fidelity: reproducing the two defects

A model that cannot reproduce the known defects proves nothing. Each round corresponds to breaking one fix of the correct design; the correct spec above is what remains once both are fixed. Each variant is a single-schema edit; the model checker is re-run with the AT goal, which must now be *found*.

Round 2 — observer notified inside the patch. Model eager notification: in `ApplyKey`, replace the deferred pair

```
pending' = pending \cup {o : OBS | watches~o = stepKey~(head~queue)}
observed' = observed
```

with an *immediate* fold of the just-installed value into the observers of that field:

```

observed' = observed \oplus
  {o : OBS | watches~o = stepKey~(head~queue) @
    o \mapsto stepVal~(head~queue)}
pending' = pending

```

Now a patch that sets a watched field and then hits a **badK** step folds the intermediate value into the observer, **AbortPatch** restores the field but cannot un-fold the observer, and at the resulting **idle** state $\text{observed } o \neq \text{fields } (\text{watches } o)$: the AT goal is *found*.

Round 1 — snapshot aliases a mutable model. Model the shallow snapshot capturing a shared reference: in **ApplyKey**, add the just-installed value to the snapshot as well, as an in-place mutation of the model the snapshot points at,

```

snap' = snap \oplus {stepKey~(head~queue) \mapsto stepVal~(head~queue)}

```

leaving the deferred notification correct. Now a patch that sets a watched field and then hits a **badK** step reaches **AbortPatch**, which restores $\text{fields}' = \text{snap}$ — but **snap** was mutated alongside the field, so the field is *not* restored, while the (correctly deferred) observer still holds the pre-patch value. At the resulting **idle** state $\text{fields } (\text{watches } o) \neq \text{observed } o$: the AT goal is *found* from the opposite side.

Under the correct design neither trace exists: **ApplyKey** defers every notification and leaves **snap** untouched, **AbortPatch** restores a value snapshot and fires nothing, and **CommitPatch** folds only final committed values. The reachability search for AT therefore returns *goal not found* for the correct design and *goal found* — the two traces above — for the corresponding buggy variant. That difference is the evidence that the model detects the defects the code was changed, round by round, to prevent.